



Objetivo arquitetural: manter uma única jornada de atendimento, preservando contexto, protocolo, histórico e progresso quando o cliente é redirecionado entre bots, áreas e atendimento humano.

1

Cliente Claro

Login autenticado e chat contínuo com a CIA, no mesmo protocolo do início ao fim.

2

Frontend React

Portal Cliente + Portal Atendente + Administração, em React / Vite / TypeScript.

3

API ASP.NET Core 8

REST, autenticação JWT, regras de negócio, DTOs e orquestração da jornada.

4

Orquestrador CIA

Decide a área responsável sem criar nova sessão. A CIA é dona do contexto.

CIA — camada central de orquestração

Os bots e as áreas não são donos do histórico. A CIA identifica o cliente, classifica a intenção, atualiza o contexto compartilhado, transfere a área e gera a resposta — sempre na mesma sessão e no mesmo protocolo *CIA-YYYYMMDD-0001*.

C — Contexto central

Problema original, tipo do problema, modem reiniciado, internet ainda fora, procedimentos realizados, pedido atual, fatos importantes e resumo persistido no PostgreSQL.

Persistência — PostgreSQL + EF Core

Sessões, mensagens (Customer / CIA / Agent), usuários, transferências de departamento, fila humana (*HumanAgentRequests*) e contexto compartilhado. Migrations incrementais, sem apagar o histórico anterior.

Triagem

Início da sessão

Suporte Técnico

Diagnóstico

Troca de Modem

Contexto herdado

Financeiro

Jornada técnica completa

Atendimento Humano

Fila + chat real



Regra de ouro: o redirecionamento muda a área responsável, mas não muda a sessão. O cliente mantém o mesmo protocolo e não repete informações já registradas.

1

Login do cliente

Perfil Customer. Cliente autenticado inicia ou retoma o atendimento em /.

2

Triagem

"Minha internet não está funcionando." O problema original é registrado no contexto.

3

Suporte Técnico

CIA pergunta apenas o que ainda não sabe. Cliente: "Já reiniciei o modem."

4

Troca de Modem

Novo bot recebe: internet fora + modem reiniciado + falha persiste. Não repregunta.

5

Financeiro

"Essa troca tem cobrança?" O financeiro recebe a jornada técnica completa.

6

Fila humana

"Quero falar com um atendente." CIA gera resumo e cria solicitação *Waiting*.

7

Atendente Claro

Funcionário assume o mesmo protocolo e conversa no mesmo chat, com todo o histórico.

O que permanece igual a cada transferência

Mesmo CustomerId

Mesma Session

Mesmo protocolo

Histórico preservado

Contexto persistido

Chat contínuo

O que muda

Apenas a **área responsável** (*CurrentDepartment*) e o registro em *DepartmentTransfer*. Após o pedido humano, o status vai para *WaitingForAgent* e, quando o funcionário assume, para *Transferred*.

Exemplo de continuidade (MVP)

Protocolo **CIA-YYYYMMDD-0001** · Cliente Lucas (**CLIENTE-001**) · Triagem → Suporte Técnico → Troca de Modem → Financeiro → Fila → Ana Souza assume e continua: "Olá Lucas, vi seu histórico. Você já realizou a reinicialização do modem."



Portal do Cliente

Perfil: Customer

Login, nome, protocolo, área atual, histórico da conversa e chat com a CIA e com o atendente.

Rotas: /login e /

Portal do Atendente

Perfil: Agent

Fila de solicitações, resumo contextual, botão Assumir atendimento e chat com o cliente.

Rota: /agent

Portal de Operação

Perfil: Admin

Visão consolidada, sessões, jornada, contexto, histórico e transbordos.

Rota: /admin

Handoff humano — continuidade real do atendimento

1

Cliente

Pede atendimento humano

2

CIA

Gera resumo + preserva histórico

3

Fila

Status: Waiting

4

Atendente

Assume a sessão

5

Chat

Cliente ↔ Atendente

O que o atendente recebe ao assumir

- Cliente e protocolo
- Problema original
- Áreas já percorridas
- Procedimentos realizados
- Fatos importantes
- Pedido atual
- Resumo do contexto
- Histórico completo de mensagens

Remetentes no mesmo chat

Customer (cliente) · **Assistant** (CIA) · **HumanAgent** (funcionário Claro). Após a fila, a CIA deixa de responder automaticamente e o diálogo continua com o atendente no mesmo protocolo.

FRONTEND

Vercel

React + Vite + TypeScript. Portais Cliente, Agent e Admin.
Variável `VITE_API_URL`.

API

Render

Docker + ASP.NET Core 8. API REST + JWT + Services.
Swagger e CORS para o frontend.

BANCO

Render PostgreSQL

EF Core + migrations incrementais. Persistência transacional da jornada e da fila humana.

Camada	Tecnologia	Responsabilidade
Interface	React / Vite / TypeScript	Login por perfil, chat, jornada, fila, painel e dashboards.
API	ASP.NET Core 8 / REST	Controllers, validação, autenticação JWT e contratos DTO.
Negócio	Services + Orchestration	Contexto central, roteamento, IA/fallback, handoff e fila humana.
Dados	EF Core + PostgreSQL	Users, sessions, messages, contexts, transfers, human agent requests.
Qualidade	xUnit + builds	Testes de contexto, jornada, handoff e fluxo humano.

Requisitos não-funcionais contemplados

Persistência real

Separação front/back

Autenticação por perfil

CORS e variáveis de ambiente

Migrations incrementais

Logs e tratamento de erros

Testes automatizados

Deploy Vercel + Render